



GA4-220501095-AA2-EV03

**BASES CONCEPTUALES ACERCA DEL LENGUAJE
UNIFICADO DE MODELADO Y PATRONES DE DISEÑO**

July Andrea Herreño

Edwin Ramirez Rodríguez

Servicio Nacional de Aprendizaje – SENA

Análisis y Desarrollo de Software

Jorge Mario Hernandez Torres

Bogotá Distrito Capital

14 de noviembre de 2025

INTRODUCCIÓN

El Lenguaje Unificado Modelado (UML) es una de las herramientas que permite a los desarrolladores documentar diferentes aspectos de un desarrollo, permitiendo así generar un mapa del funcionamiento de un sistema en diferentes ámbitos, no solo en con un programa en producción sino los pasos previos a su construcción como con los diagramas de casos de uso, por ejemplo. Es así que en el presente documento se realiza un glosario con las terminologías mas importantes que se usan en varias instancias del modelado de diagramas con UML con el fin de generar una familiaridad con los tecnicismos y así apropiarlos cuando se necesite emplear algún diagrama en particular.

OBJETIVOS

1. Definir de manera breve que es el Lenguaje Unificado Modelado (UML) y realizar un glosario completo en el que se explique de manera breve y simple cada una de las terminologías mas importantes que son usadas con el modelado de diagramas por medio de este lenguaje.
2. Apropiar los términos, de manera que sea posible identificarlos y poder comprender su utilidad al momento de plantear soluciones por medio de las diferentes posibilidades de diagramas con las reglas del UML.

¿QUÉ ES EL LENGUAJE UNIFICADO DE MODELADO UML?

El Lenguaje Unificado de Modelado (UML) es un lenguaje de modelado estandarizado de propósito general en el campo de la ingeniería de software. Este estándar es gestionado y creado por el Object Management Group (OMG). UML incluye un conjunto de técnicas de notación gráfica para crear modelos visuales de sistemas de software complejos. El UML se utiliza para especificar, visualizar, modificar, construir y documentar los artefactos de un sistema de software complejo orientado a objetos en desarrollo. UML ofrece una forma estándar de visualizar los planos arquitectónicos de un sistema, incluyendo elementos como: actividades, actores, procesos de negocio y esquemas de bases de datos.

GLOSARIO

- **Clase abstracta:** una clase que no se puede instanciar porque no se ha especificado una implementación completa para todas sus operaciones. Tipo de dato abstracto.
- **Tipo de datos abstracto:** una clase creada para encapsular un tipo de información común a una aplicación. Se suele usar para elementos como direcciones, que contienen varios campos individuales combinados de una forma específica y editados según reglas específicas propias de esa combinación.
- **Operación abstracta:** una firma de operación sin una implementación asociada.
- **Abstracción:** una representación única de una entidad del mundo real. El objetivo de esta representación no es ofrecer una descripción exhaustiva, sino una descripción útil para una aplicación o propósito específico.
- **Acción:** es una respuesta a un evento en un diagrama de estados, generalmente parte de la transición de un estado a otro y generalmente atómica (no se puede dividir en subtarefas).
- **Estado de acción:** es un estado de objeto que representa la ejecución de una acción, normalmente la invocación de una operación.
- **Activación:** es la ejecución de una operación sobre un objeto u otra llamada a un objeto para iniciar un comportamiento sobre ese objeto.
- **Objeto activo:** es un objeto que controla un hilo o proceso y que puede iniciar actividades de control. En un diagrama de secuencia, se identifica ampliando la línea de tiempo vertical hasta convertirla en un rectángulo estrecho. En un diagrama de objetos, se indica resaltando el icono del objeto en negrita.

- **Actividad:** es el procesamiento que realiza un objeto en un estado específico. Una actividad suele ser no atómica, es decir, puede estar compuesta por cualquier número de subtarefas.
- **Actor:** una persona, sistema o dispositivo que interactúa con un sistema en un diagrama de casos de uso.
- **Agente:** un objeto que puede ser origen y destino de una interacción.
- **Agregar:** una clase que representa el todo en una relación todo-parte.
- **Agregación:** es una asociación de tipos asimétrica que se utiliza para modelar una relación todo-parte entre clases. Generalmente se usa para modelar ensamblajes de objetos para crear nuevos objetos. Véase también: Composición
- **Antepasado:** es cualquier clase que se encuentre por encima de la clase de referencia en una jerarquía de generalización. Véase también: Superclase
- **Arquitectura:** la arquitectura se refiere a la forma en que los componentes encajan entre sí. Puede utilizarse para describir cualquier sistema complejo, por ejemplo, "arquitectura de software" o "arquitectura de red".
- **ARP:** el Protocolo de Resolución de Direcciones (ARP) forma parte de la capa de Internet. Obtiene las direcciones de hardware de los hosts ubicados en la misma red física.
- **Artefacto:** en el Lenguaje Unificado de Modelado (UML), un artefacto es la especificación de una pieza física de información que se utiliza en un proceso de desarrollo de software o en el despliegue y operación de un sistema.
- **Asociación:** una relación entre clases que especifica el tipo de enlaces que se pueden crear entre objetos de las clases asociadas.
- **Clase de asociación:** información sobre una asociación que se ha abstraído en su propia clase.
- **Supuestos:** comportamientos que deben ser manejados por otro caso de uso antes de permitir que alguien acceda al caso de uso en cuestión.
- **Evento o acción asíncrona:** es un tipo de mensaje que no requiere respuesta ni que el objeto remitente tenga que esperar.
- **Atributo:** es un miembro de clase utilizado para definir una unidad de información propiedad de la clase contenedora, incluyendo detalles como el tipo y el dominio de los valores.
- **Transición automática:** es una transición desencadenada por la finalización o terminación de una actividad de estado. Véase también: Transición

- **Evento de resistencia:** tipo de mensaje en el que el cliente solo puede enviar un mensaje si el proveedor está listo para recibirlo de inmediato. Si el proveedor no está listo, el cliente descarta el mensaje.
- **Clase base:** es una clase raíz en una jerarquía de generalización.
- **Elemento enlazado:** una clase resultante de la especificación de parámetros en una clase plantilla.
- **Categoría:** en una vista lógica, una categoría es una forma de paquete; encapsula clases.
- **Clase:** una abstracción de un conjunto de información, comportamientos y relaciones relacionadas que describe una entidad. Una definición utilizada para crear objetos. Véase también: Objeto
- **Método de clase:** una operación o método específico de una clase, en lugar de aplicarse a los objetos de esa clase.
- **Variable de clase:** un atributo que es específico de una clase en lugar de los objetos de la clase.
- **Clasificación:** es el proceso de encontrar información, comportamientos y relaciones comunes entre objetos. Véase también: Clase y Objeto
- **Cliente:** es un objeto que inicia una solicitud.
- **Cohesión:** la cohesión mide el grado en que todas las partes de un objeto contribuyen a un mismo propósito. Una alta cohesión implica que todos los elementos del objeto contribuyen al mismo propósito. Una baja cohesión implica que los distintos elementos contribuyen a propósitos diferentes.
- **Colaboración:** es una interacción entre objetos.
- **Componente:** es una unidad física de software que puede residir en la memoria de un procesador y que realiza (implementa) un conjunto de interfaces.
- **Vista de componentes:** es la vista dedicada a la descripción de las unidades de implementación de software; puede utilizarse en combinación con la vista de despliegue.
- **Compuesto:** una relación todo-parte de tipo composición entre clases.
- **Estado compuesto:** un estado compuesto puede tener muchas regiones, cada una de las cuales puede contener subestados. Un estado compuesto con más de una región se denomina a veces estado compuesto ortogonal.
- **Composición:** es un caso especial de la relación parte-todo entre clases donde la vida de la parte depende de la vida del todo. Cuando el todo se extingue, la parte deja de existir.
- **Clase de hormigón:** una clase que puede instanciarse porque se ha especificado una implementación (método) para cada operación heredada o propia de la clase. Véase también: Clase abstracta

- **Concurrencia:** la ocurrencia de múltiples actividades al mismo tiempo, ya sea mediante la ejecución de múltiples hilos o mediante múltiples procesos.
- **Restricción:** la extensión UML de la definición de un elemento del modelo que permite agregar nuevas reglas o modificar las existentes.
- **Contexto:** el conjunto de elementos del modelo que define el marco de referencia para la interpretación e implementación de un elemento del modelo en particular.
- **Contrato:** un acuerdo entre clases sobre la gestión de responsabilidades y la implementación en las definiciones de clase.
- **Enganche:** una medida del grado de dependencia entre los elementos del modelo.
- **Diccionario de datos:** un diccionario de datos define el vocabulario del dominio del problema. Las entradas del diccionario de datos pueden cambiar a lo largo del proyecto, pero solo deben modificarse con el consentimiento de todos los participantes. Utilice un diccionario de datos para garantizar una comunicación coherente y de alta calidad.
- **Tipo de datos:** el formato de la información permitía un atributo, por ejemplo, entero, dirección, carácter.
- **Descomposición:** separar una entidad o proceso en unidades funcionales más pequeñas.
- **Delegación:** delegar la responsabilidad de una tarea a otra entidad, generalmente ocultando la implementación de la tarea al solicitante.
- **Dependencia:** una relación entre dos entidades del modelo en la que un cambio en una (el elemento independiente del modelo) afectará a la otra (el elemento dependiente del modelo).
- **Vista de despliegue:** una vista dedicada a la descripción de arquitecturas de hardware; puede utilizarse en combinación con la vista de componentes.
- **Asociación derivada:** una asociación creada mediante la combinación de varias asociaciones.
- **Atributo derivado:** un atributo cuyo valor puede determinarse examinando uno o más valores de otros atributos.
- **Dominio:** un área temática con su vocabulario y semántica asociados.
- **Elemento:** unidad de información utilizada para construir un modelo.
- **Encapsulación:** la capacidad de proporcionar a los usuarios una interfaz bien definida para un conjunto de funciones, ocultando su funcionamiento interno. En programación orientada a objetos, la técnica de mantener agrupadas las estructuras de datos y los métodos (procedimientos) que actúan sobre ellas.
- **Evento:** un estímulo sistémico, a menudo en forma de mensaje de un objeto a otro.

- **Flujo excepcional de acontecimientos:** una condición que altera el flujo normal de control en un sistema.
- **Factorización:** el proceso de encontrar y extraer elementos comunes entre clases. Véase también: Generalización.
- **Generalización:** 1. El proceso de factorización de clases para definir una jerarquía de elementos compartidos organizados en clases. 2. La jerarquía de clases resultante del proceso de factorización. 3. Un tipo de asociación utilizada para relacionar clases en una jerarquía de clases. Véase también: Herencia.
- **Condición de guardia:** una condición asociada a una transición de estado que debe ser verdadera antes de que pueda tener lugar la transición.
- **Herencia de implementación:** compartir tanto la interfaz como la implementación de un método. Véase también: Herencia de interfaces
- **Importar:** una dependencia entre paquetes donde un paquete obtiene acceso al contenido del otro agregando el contenido del paquete importado al suyo propio.
- **Herencia:** una relación entre clases que permite compartir elementos. Véase también: Generalización.
- **Instancia:** la implementación de una clase como un objeto.
- **Método de instancia:** un método exclusivo de un objeto. Véase también: Método de clase
- **Variable de instancia:** un atributo que es único para un objeto.
- **Interacción:** comunicación entre objetos.
- **Interfaz:** la parte visible de una clase. Se utiliza normalmente para describir las firmas públicas de las operaciones en una clase.
- **Herencia de interfaz:** compartir únicamente la interfaz o la declaración de un elemento, no la implementación.
- **Iteración:** repetir una función o conjunto de funciones más de una vez. La repetición puede aplicarse a un valor, un recuento, un tiempo, la finalización de una tarea o cualquier otro tipo de condición.
- **Enlace:** relación entre dos objetos. Véase también: Asociación
- **Atributo de enlace:** unidad de información sobre un enlace. Véase también: Clase de asociación
- **Vista lógica:** una vista dedicada a la descripción de las opciones de diseño para una solución de software.
- **Flujo principal de eventos:** el flujo normal de control en un sistema.

- **Fusión del control:** para sincronizar o coordinar la finalización de múltiples hilos o procesos y transferir el control a un único hilo o proceso. La fusión del control no requiere la finalización de todos los hilos o procesos. La fusión puede estar condicionada a la finalización de uno o más hilos, al tiempo o a otras condiciones.
- **Mensaje:** comunicación entre objetos.
- **Metamodelo:** 1. La especificación de un lenguaje para describir modelos orientados a objetos. 2. Un modelo que describe los elementos, y las relaciones entre esos elementos, que conforman un dominio especificado.
- **Método:** la ejecución de una operación.
- **Metodología:** un conjunto organizado y documentado de procedimientos y directrices para una o más fases del ciclo de vida del software, como el análisis o el diseño. Muchas metodologías incluyen una notación diagramática para documentar los resultados del procedimiento; un enfoque paso a paso, tipo «receta», para llevar a cabo el procedimiento; y un conjunto de criterios objetivos (idealmente cuantificados) para determinar si los resultados del procedimiento son de calidad aceptable.
- **Modelo:** una representación de un dominio en otra forma, generalmente diseñada para simplificar y facilitar la comprensión del dominio.
- **Elemento del modelo:** la unidad mínima de definición semántica en un modelo. Un mismo elemento puede aparecer en varios diagramas. Por ejemplo, un evento aparece en diagramas de secuencia, diagramas de colaboración y diagramas de estado. En los tres contextos, proporciona la misma información semántica.
- **Multiplicidad:** una restricción sobre el número de objetos que pueden participar en un extremo de una asociación.
- **Navegabilidad:** una especificación de la dirección de comunicación permitida a lo largo de una asociación.
- **Nodo:** una unidad de hardware capaz de contener y ejecutar software. Véase también: Vista de despliegue.
- **Notación:** cualquier sistema particular de caracteres, símbolos o expresiones abreviadas utilizado en el arte o la ciencia para expresar brevemente hechos técnicos, cantidades, etc. Especialmente, el sistema de cifras, letras y signos utilizado en aritmética y álgebra para expresar números, cantidades u operaciones.

- **Objeto:** una entidad identificable de forma única, compuesta por información, comportamientos y relaciones con otras entidades.
- **Activación de objetos:** un objeto que controla un hilo o proceso y que puede iniciar actividades de control. En un diagrama de secuencia, se identifica ampliando la línea de tiempo vertical hasta convertirla en un rectángulo estrecho. En un diagrama de objetos, se indica resaltando el icono del objeto en negrita. Véase también: Objeto activo.
- **Lenguaje de restricciones de objetos:** un lenguaje formal utilizado para expresar los efectos secundarios de las restricciones en los modelos UML.
- **Grupo de gestión de objetos:** el grupo de estándares que actualmente supervisa el desarrollo y la aceptación de estándares relacionados con el desarrollo de software orientado a objetos.
- **Terminación del objeto:** el final del ciclo de vida de un objeto, que generalmente indica su destrucción. Se identifica con una X grande en la parte inferior de la línea de tiempo del diagrama de secuencia del objeto en cuestión.
- **Grupo de trabajo de análisis y diseño orientado a objetos:** el grupo asignado por la OMG para generar y supervisar la solicitud de propuestas (RFP) para un metamodelo para el diseño de objetos.
- **Operación:** declaración de una unidad de comportamiento dentro de una clase. Véase también: Método
- **Sobrecarga:** se utiliza para describir métodos cuyo nombre se comparte, pero cuyos argumentos varían en número o tipo. Véase también: Polimorfismo.
- **Paquete:** un mecanismo de propósito general para agrupar modelos y elementos de modelos, normalmente por funciones o usos similares dentro del contexto de un sistema. Véase también: Componente
- **Patrón:** una solución de diseño recurrente que contiene un conjunto de clases que cooperan entre sí.
- **Polimorfismo:** se utiliza para referirse a métodos cuya firma es la misma pero cuya implementación es diferente. Véase también: Sobrecarga
- **Neurona postsináptica:** una neurona, cuyo cuerpo celular o dendrita es el cual recibe un impulso eléctrico a través de una hendidura sináptica mediante la liberación de un neurotransmisor químico desde la terminal axónica de una neurona presináptica.
- **Post-condición:** una condición que debe ser verdadera al finalizar la tarea en cuestión.

- **Condición previa:** una condición que debe ser verdadera antes de la ejecución de la tarea propietaria.
- **Tipo de datos primitivo:** la unidad mínima de definición de datos proporcionada por el entorno de implementación. Véase también: Tipo de dato abstracto
- **Privado:** una forma de restricción de visibilidad que impide el acceso a un elemento del modelo desde fuera de la clase propietaria. Véase también: Público
- **Enunciado del problema:** una descripción concisa del problema o desafío del proyecto.
- **Proceso:** una línea de acción específica destinada a lograr un resultado.
- **Propiedad:** un valor con nombre que describe una característica de un elemento.
- **Protegido:** un tipo de visibilidad de elementos del modelo que permite que solo las subclases tengan acceso al elemento del modelo protegido, por ejemplo, una operación o un atributo.
- **Público:** una forma de restricción de visibilidad que permite el acceso a un elemento del modelo desde fuera de la clase propietaria. Véase también: Privado
- **Índice:** un elemento de modelado de asociaciones que identifica uno o más atributos descriptivos utilizados para navegar por una asociación. Es sinónimo de claves o índices en una base de datos o en el dominio del modelado de datos.
- **Realización:** cuando una clase implementa una interfaz especificada por otra clase (o interfaz, como en Java).
- **Asociación reflexiva:** una asociación que relaciona objetos de la misma clase entre sí. Véase también: Asociación.
- **Ingeniería inversa:** el proceso de traducir el código en elementos de modelado.
- **Role:** una forma de especificar la naturaleza de la participación de un objeto en una asociación.
- **Guion:** un único camino lógico a través de un caso de uso, a menudo utilizado como caso de prueba. Véase también: Vista de caso de uso
- **Enviar evento:** un mensaje o señal enviado a otro objeto desde dentro de un diagrama de estados.
- **Servidor:** un objeto que actúa únicamente como receptor en una interacción. Nótese que el mismo objeto puede actuar como cliente en otra interacción.
- **Señal:** un estímulo asíncrono enviado de un objeto a otro.
- **Firma:** la interfaz de una operación se describe mediante el nombre, la cantidad, el tipo y el orden de los argumentos, y a veces también mediante el tipo de datos de retorno (la interpretación varía según el lenguaje de implementación). Véase también: Polimorfismo, Sobrecarga e Interfaz.
- **Especialización:** lo opuesto a la generalización. Véase también: Generalización y factorización.

- **División del control:** iniciar múltiples hilos o procesos desde un solo hilo o proceso.
- **Estado:** el estado de un objeto en un momento dado.
- **Estereotipo:** una extensión UML que proporciona un medio para describir o calificar con mayor detalle un elemento del modelo sin definir su implementación.
- **Subclase:** una clase especializada conectada a una clase más general o "clase padre" en una jerarquía de clases de generalización.
- **Subestado:** un estado que forma parte de o está contenido dentro de otro estado (estado compuesto o superestado).
- **Superclase:** una clase generalizada conectada a una clase más especializada o "clase hija" en una jerarquía de clases de generalización.
- **Superestado:** un estado que contiene otros estados, por ejemplo, un estado definido en un nivel de abstracción superior.
- **Evento o acción síncrona:** una forma de comunicación que requiere un acuse de recibo de la solicitud.
- **Sistema:** el término "sistema" se refiere al conjunto de todas las funciones y recursos que apoyan el objetivo de los usuarios.
- **Valor etiquetado:** una extensión de las propiedades de un elemento UML que permite agregar nueva información a la especificación de un elemento.
- **Plantilla o clase de plantilla:** una clase parametrizada que sirve como modelo para la construcción de otras clases.
- **Evento temporal:** un evento que especifica la cantidad de tiempo transcurrido desde que se accedió al estado actual.
- **Transición:** un cambio de un estado a otro dentro de un objeto.
- **Evento desencadenante:** evento que inicia el caso de uso. El evento podría ser un momento puntual (por ejemplo, el final del mes), una condición del sistema (por ejemplo, una cuenta está sobregirada) o una solicitud de un actor (por ejemplo, seleccionar una opción).

CONCLUSIÓN

1. El lenguaje UML es una herramienta que en el desarrollo de software facilita comprender el funcionamiento que se espera de un programa de acuerdo a las necesidades del negocio, es por ello que a pesar de no ser visto dentro de los elementos clave del desarrollo, al menos en las grandes compañías se hace fundamental para que todos o la gran mayoría de los eslabones de la organización puedan comprender como esta concebido el negocio, así mismo los desarrolladores puedan comprender la estructura existente y así saber donde se pueden aplicar mantenimientos, optimizaciones, nuevos desarrollos sin afectar al negocio.
2. Los modelos UML en una fase de aprendizaje se hacen fundamentales para entender cómo se concibe un sistema y para que se concibe el sistema, con diferentes esquemas de diagramas se pueden comprender y entrelazar diferentes instancias del desarrollo, pasando desde un diagrama de casos de uso, por un diagrama de clases, hasta un diagrama de despliegue. Si bien esta puede ser una tarea tediosa, en el mundo del desarrollo, la documentación juega un rol menospreciado, pero a su vez fundamental, un buen sistema no solo se compone de un código limpio y escalable, también lo hace desde una estructuración lógica por medio del UML.

BIBLIOGRAFÍA

- Booch, G. (1994). Object-oriented analysis and design. Redwood City.
- Booch, G., Rumbaugh, J., Jacobson, I., Molina, J. J. G., & Martínez, J. S. (2006). El lenguaje unificado de modelado: guía del usuario. Addison-Wesley.
- Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley Professional.
- Cohn, M. (2018). User Stories and User Story Examples by Mike Cohn. [online] Mountain Goat Software.<https://www.mountaingoatsoftware.com/agile/user-stories>
- Cuevas Agustín, Gonzalo. (1991). Ingenieria del Software.Práctica de la programación, Editorial RA-MA, 1 impresión.
- Flores Cueto, J. J., & Bertolotti Zúñiga, C. (2013). Diagrama de clases en uml. 6.
- Gutiérrez, J. (s.f.). Diagramas UML de casos de uso y de requisitos.
- Larman, C. (2002). "Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process". 2nd Ed. Prentice Hall.
- Norris & Rigby. (1994). "Ingeniería de software explicada", 1 edición Editorial Megabyte-Noriega editores, México.
- Pressman, Roger. (1998). Ingeniería del Software. Un enfoque práctico, Editorial Mc. Graw-Hill, 4ta edición, 1998.
- Rumbaugh, J., Jacobson, I., & Booch, G. (2004). El Lenguaje Unificado de Modelado. Elements, 30.
- Schmuller, J. (2001). Aprendiendo UML en 24 horas. Pearson Educación.
- Schmuller, J., & Garza Marín, A. D. (2000). Aprendiendo UML en 24 horas.
- Wake, W.C. (2003). "Extreme Programming Explored" y "Refactoring Workbook"